



JIRA, AGILE & SDLC

QA Workflows from Ideas Through Reliable Releases

This program covers Agile, SDLC, JIRA, test case design, manual testing, and automation basics — one concept at a time.

Agile & SDLC Basics

Understand software development lifecycles and iterative Agile delivery methods.

JIRA Fundamentals

Navigate projects, issues, boards, and backlogs to track and manage QA work.

Test Case Design & Test Planning

Build repeatable test cases and structured plans covering scope, risks, and resources.

QA Lifecycle & Manual Testing

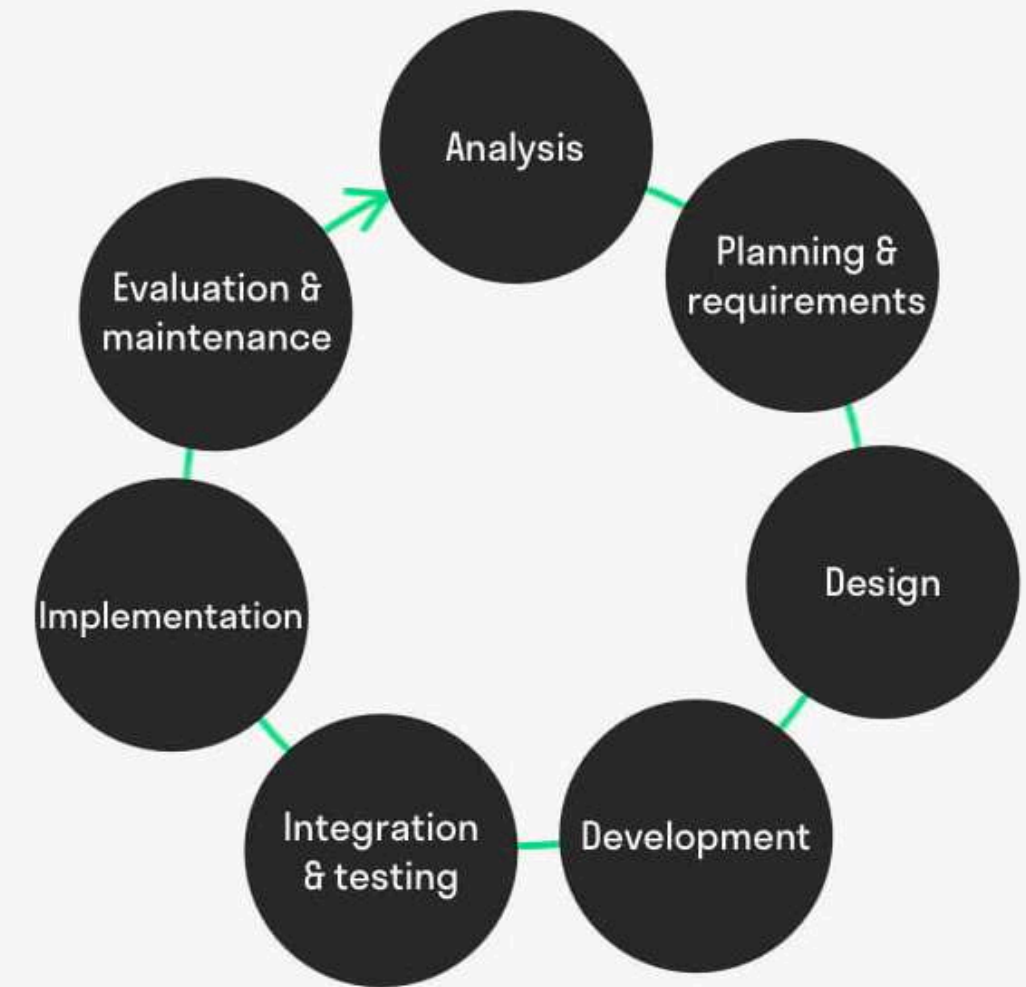
Execute tests, manage defects, and apply exploratory and regression techniques.

LEARNING ROADMAP

Your Training Journey: From Basics to Automation

SDLC OVERVIEW

How software moves from idea to retirement



Planning & Design

Defines goals, users, scope, and risks. Describes architecture, interfaces, and data flows.

Development & Testing

Turns approved designs into working software. Verifies quality and expected behavior before release.

Maintenance

Improves and supports software after release. Manages updates until the system is decommissioned.

HOW SOFTWARE IS BUILT?

From idea to release and maintenance

1. PLANNING
WHAT ARE WE BUILDING?
WHO NEEDS IT? WHAT ARE
THE GOALS?

2. DESIGN
HOW WILL THE SYSTEM
WORK? WHAT WILL THE
SCREENS, DATA, AND
ARCHITECTURE LOOK LIKE?

3. DEVELOPMENT
DEVELOPERS BUILD THE
SOFTWARE.

4. TESTING
QA CHECKS WHETHER THE
SOFTWARE WORKS AS
EXPECTED.

5. RELEASE
THE COMPLETED
SOFTWARE IS DELIVERED
TO USERS.

6. MAINTENANCE
THE TEAM FIXES PROBLEMS
AND IMPROVES THE
SOFTWARE AFTER RELEASE.

Agile delivers useful software in small increments. Short iterations expose problems earlier in the process.

Teams welcome feedback throughout every stage of development. Priorities can change between iterations as needs evolve.

Working software demonstrates real, measurable progress. Collaboration keeps business and technical teams aligned.

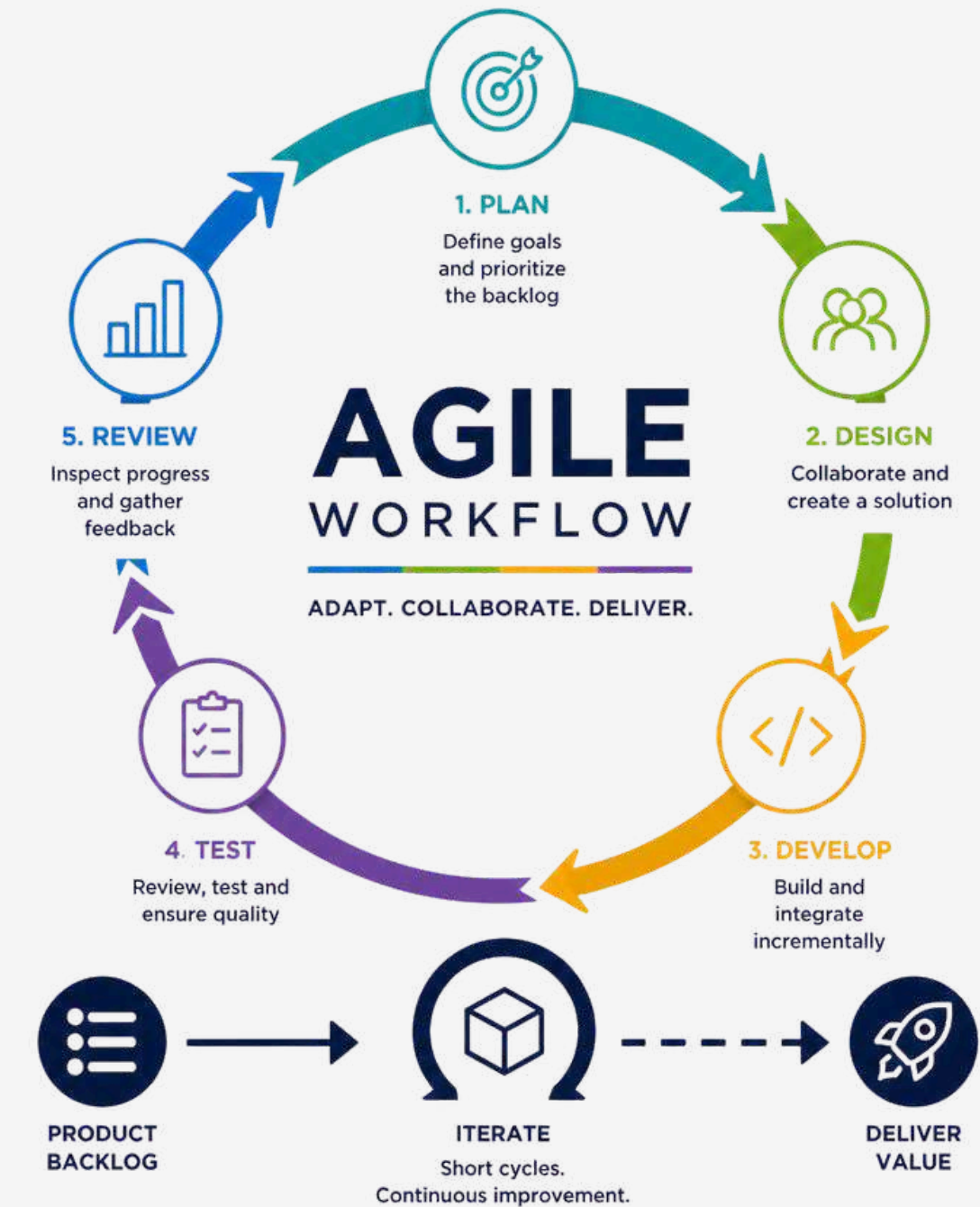
Incremental delivery reduces risk and builds confidence. Continuous improvement drives quality in every sprint.

AGILE BASICS

Incremental Delivery & Collaboration

AGILE WORKFLOW

How Work Moves Through a Sprint



Sprint Start

- Product backlog stores prioritized work items
- Sprint planning selects achievable backlog tasks
- Daily standups reveal progress and blockers

During the Sprint

- Development and testing happen throughout
- Team collaborates to resolve blockers quickly
- Progress tracked on the sprint board daily

Sprint Close

- Sprint reviews collect stakeholder feedback
- Retrospectives improve the next sprint
- Completed work is released or queued for release

AGILE: BUILD, TEST, LEARN, IMPROVE

How teams deliver software in small steps

Simple flow

Plan → Build → Test → Review → Improve → Repeat

Work in small pieces

Break large features into smaller deliverable work.

Deliver frequently

Complete useful work in short iterations.

Get feedback

Show working software and learn from users and stakeholders.

Adapt to change

Priorities can change as needs change.

Work together

Developers, QA, product, and business teams collaborate.

Break It Down

Divide the online shopping feature into small, deliverable increments (e.g., browse, add to cart, checkout, payment).

Prioritize by Value

Rank each increment by customer value. Which delivers the most benefit first? Place highest-value items at the top of the backlog.

Plan Your Sprint

Select increments that fit one short sprint. Consider team capacity. Commit only to work that can realistically be completed and tested.

Feedback & Sharing

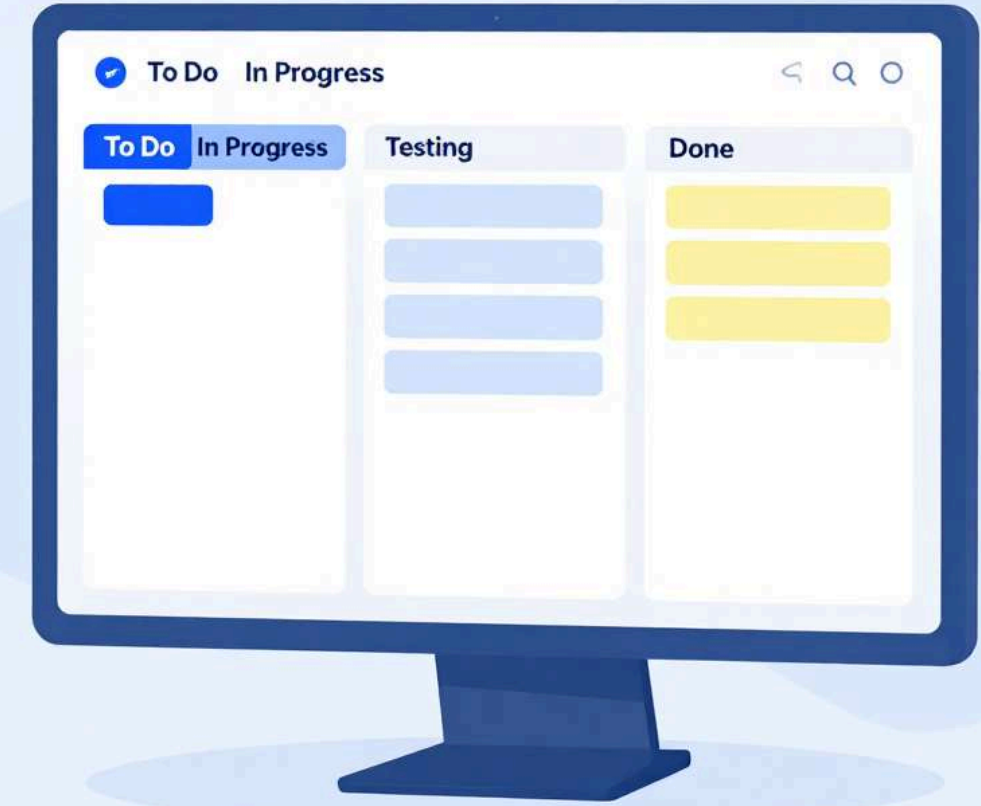
Explain how customer feedback could shift priorities between sprints. Present your sprint plan to the group and discuss trade-offs.

PRACTICE AGILE THINKING

Apply Agile to a Real Feature

JIRA BASICS

Tracking Tasks, Bugs, Stories & Progress



Projects & Issues

Projects organize work for specific teams. Issues represent individual units of work — tasks, bugs, or stories — each with a status, priority, and assignee.

Boards & Backlogs

Boards visualize work across workflow stages at a glance. Backlogs store upcoming prioritized issues waiting to be pulled into a sprint.

Filters & Search

Filters find issues using selected conditions such as assignee, status, or label. Saved filters help teams monitor recurring work patterns quickly.

WHERE THE TEAM TRACKS WORK?

Create, assign, track, and manage work

Simple example

Project → Issue → Assignee → Status → Completion

Project

A place where a team's work is organized.

Issue

A single piece of work, such as a story, task, or bug.

Board

A visual view showing where work currently is.

Backlog

A list of future work waiting to be prioritized.

Assignee

The person responsible for the issue.

Priority

How important or urgent the issue is



ISSUE WORKFLOW

How Issues Move Through JIRA Statuses



To Do: unstarted issues awaiting action.
In Progress: actively worked by an assignee.
Testing: awaiting quality verification.
Done: completed and accepted issues.
Assignees own the next required action.
Status changes record all work movement.

EVERY ISSUE MOVES THROUGH STATUSES AS WORK PROGRESSES

TO DO → IN PROGRESS → TESTING → DONE

TO DO

The issue is created but work has not started.

IN PROGRESS

Someone is actively working on the issue.

TESTING

The work is ready for QA to verify.

DONE

The work is completed and accepted.

Important note

Assignee = person responsible for the next action.



Create a Story

Create a new issue for a password reset feature. Add a clear summary and a detailed description of the expected behavior.

Configure the Issue

Set the issue priority (e.g., Medium or High). Assign an owner responsible for completing the work.

Move Through Statuses

Transition the issue from To Do → In Progress → Testing → Done. Observe how the board updates at each stage.

Comment & Filter

Add a comment describing your progress. Use JIRA filters to locate the issue by assignee, priority, or status.

PRACTICE JIRA NAVIGATION

Build Confidence with a Real JIRA Task

USER STORY STRUCTURE

Writing Stories from the User's Perspective



The Formula

As a [user or role], I want [one capability], so that [expected value].

Name the user clearly. Describe one desired capability. Explain the outcome it delivers.

Example Story

As a registered customer, I want to reset my password, so that I can regain access to my account without contacting support.

Keep stories small and focused on one need.

Quality Checks

Each story must be small enough to complete in one sprint, testable with clear acceptance criteria, and valuable to a real user or stakeholder.

Independent

Stories avoid unnecessary dependencies on other stories

Negotiable

Stories allow collaborative solutions between team members

STRONG STORY CHECKLIST

Evaluate Story Quality Using INVEST Principles

Valuable

Stories deliver meaningful outcomes to the end user

Estimable

Stories have clear, understandable scope for the team

WHAT DOES THE USER NEED?

Describe a feature from the user's point of view

A user story explains WHO needs WHAT and WHY.

Formula

As a [user],
I want [something],
so that [benefit].

Example

As a registered customer,
I want to reset my password,
so that I can regain access to
my account.

A good user story should be

Clear : everyone understands the need.

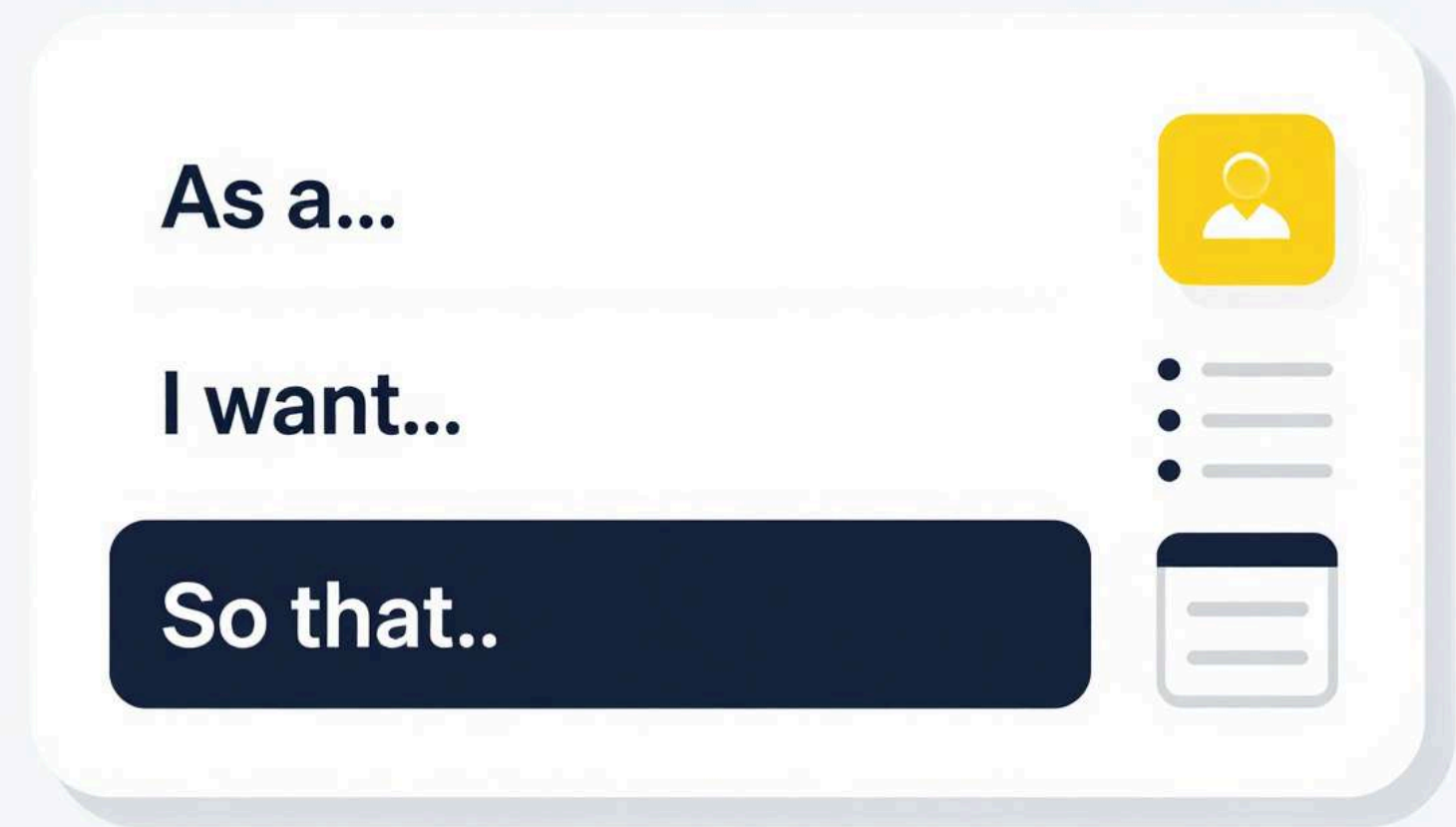
Small: focused on one requirement.

Valuable: provides value to the user.

Testable : QA can verify it.

PRACTICE USER STORIES

Write & Review a Login User
Story



As a...

I want...

So that..

Identify the User

Choose a familiar login feature. Who needs access? Name the user role clearly — e.g., registered customer or admin.

Write the Story

Use the format: As a [user], I want [capability], so that [value]. State one desired action and explain why it matters.

Review with INVEST

Check: Is it Independent, Negotiable, Valuable, Estimable, Small, and Testable? Refine until all criteria are met.



ACCEPTANCE CRITERIA

Defining Successful Story Completion

- Criteria state observable expected behavior.
- Guide development and testing decisions.
- Include positive, negative, and boundary conditions.
- Avoid vague words like "fast" or "easy".
- Agree criteria before development begins.

GIVEN

Describes the starting context or precondition before the user acts.

Successful Login

Given a registered user on the login page, When they enter valid credentials, Then they are redirected to the dashboard.

WHEN / THEN

When describes the user action. Then describes the expected observable outcome.

Incorrect Password

Given a user on the login page, When they enter a wrong password, Then an error message is displayed and access is denied.

GIVEN WHEN THEN

Acceptance Criteria in Practice

WHAT DOES “DONE” MEAN?

Clear conditions that tell us whether a story is successful

Acceptance criteria should:

Define expected behavior
What should the system do?

Be observable

We should be able to see or verify the result.

Cover different situations

Include positive, negative, and boundary cases.

Be agreed before development

Developer and QA should understand the expected result.

GIVEN

What is the starting situation?

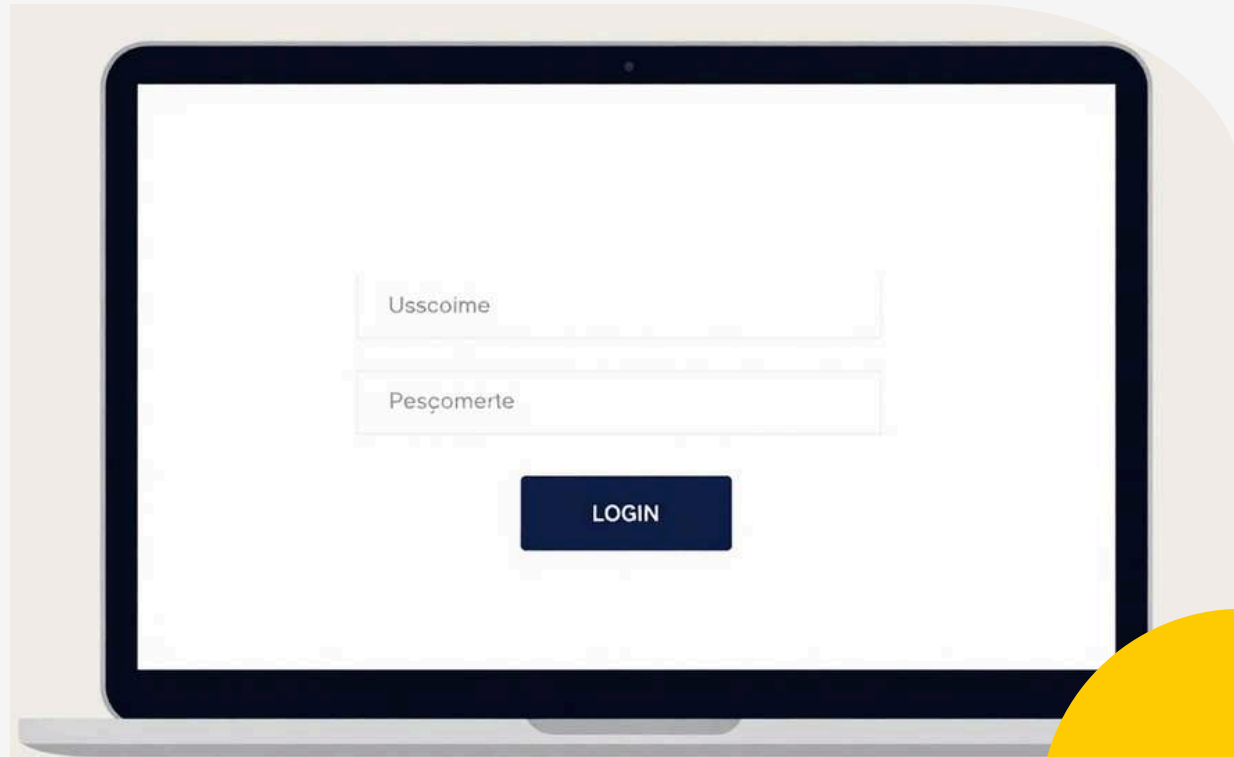
WHEN

What does the user do?

THEN

What should happen?

Acceptance criteria tell developers what to build and testers what to verify.



PRACTICE LOGIN TESTING

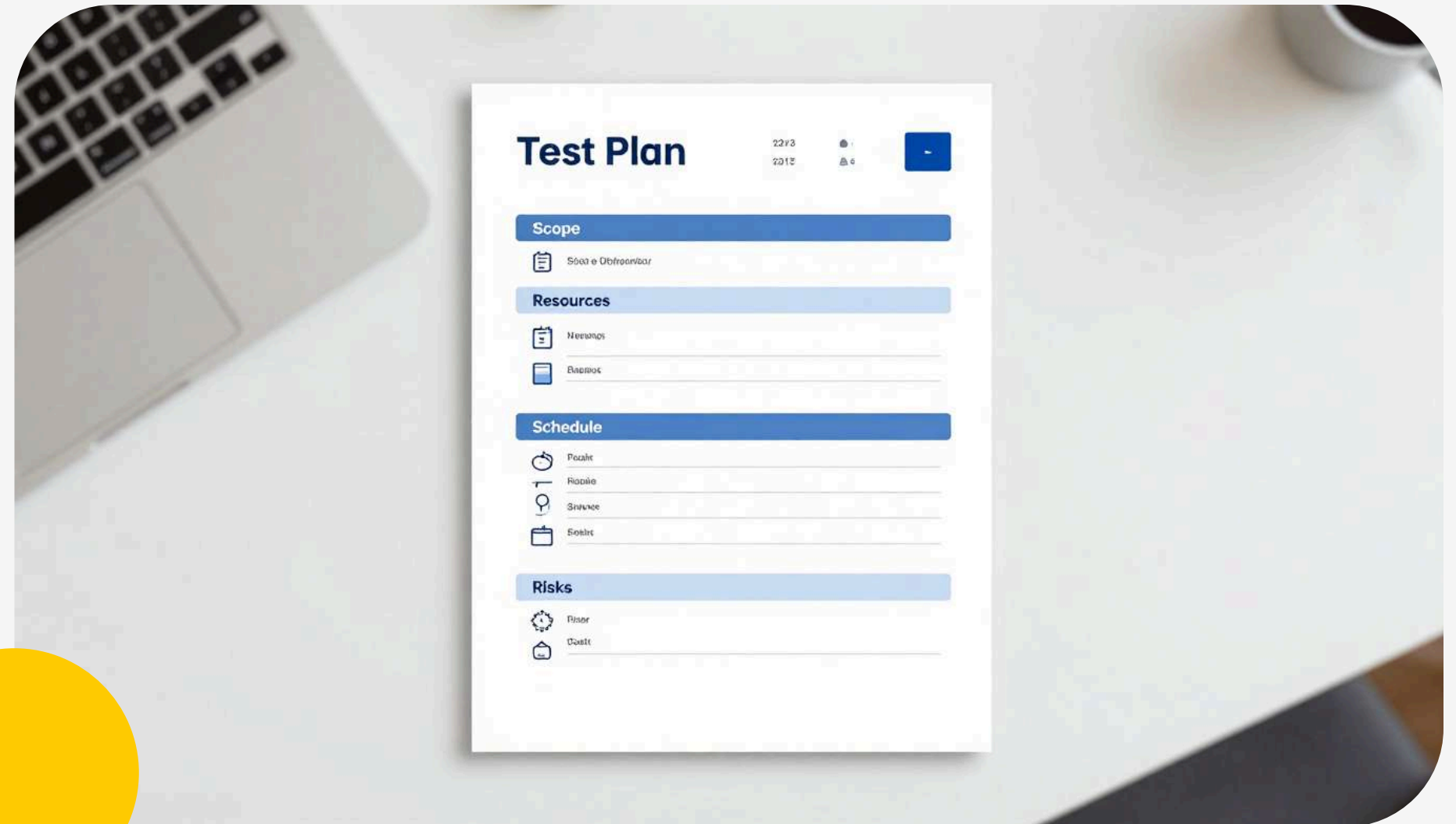
Apply Test Design to a Login
Feature

- Identify positive and negative scenarios
- Include empty and boundary values
- Write steps and expected results
- Assign each case a priority
- Compare cases with a partner



TEST PLAN ESSENTIALS

Organizing Testing Before Execution



Objectives & Scope

Objectives state what testing must prove. Scope identifies included and excluded features. Together they set clear boundaries for the entire test effort.

Resources & Schedule

Resources specify people, tools, and environments needed. Schedule defines activities and deadlines. Assign owners to each planned activity.

Risks & Mitigation

Identify risks that could affect quality or timelines. Define mitigation steps to reduce likelihood. Prepare contingency actions if risks occur.

HOW WILL WE TEST THE FEATURE?

Plan the testing before execution begins

1. Scope

What are we testing?

2. Objectives

What must the testing prove?

3. Test Approach

Which types of testing will we perform?

4. People & Responsibilities

Who will test and who will review?

5. Tools & Environment

What tools, systems, and test data are needed?

6. Schedule

When will testing happen?

7. Risks & Mitigation

What could go wrong, and how will we reduce the risk?

TEST CASE ANATOMY

Components of Repeatable Verification Steps

Standardized Test Case

Preconditions	Steps				
	Type	Desc	Expected	Actual	Pass/Fail
1. Test ID					
2. Objective					
3. Scope					
4. Environment					
5. Data					
6. Steps					
7. Expected					
8. Actual					
9. Pass/Fail					
10. Notes					
11. Date					
12. Tester					
13. Version					
14. Comments					
15. Status					
16. Priority					
17. Assigned To					
18. Assigned Date					
19. Completed Date					
20. Closed Date					
21. Closed By					
22. Closed Reason					
23. Closed Status					
24. Closed Date					
25. Closed By					
26. Closed Reason					
27. Closed Status					
28. Closed Date					
29. Closed By					
30. Closed Reason					
31. Closed Status					
32. Closed Date					
33. Closed By					
34. Closed Reason					
35. Closed Status					
36. Closed Date					
37. Closed By					
38. Closed Reason					
39. Closed Status					
40. Closed Date					
41. Closed By					
42. Closed Reason					
43. Closed Status					
44. Closed Date					
45. Closed By					
46. Closed Reason					
47. Closed Status					
48. Closed Date					
49. Closed By					
50. Closed Reason					
51. Closed Status					
52. Closed Date					
53. Closed By					
54. Closed Reason					
55. Closed Status					
56. Closed Date					
57. Closed By					
58. Closed Reason					
59. Closed Status					
60. Closed Date					
61. Closed By					
62. Closed Reason					
63. Closed Status					
64. Closed Date					
65. Closed By					
66. Closed Reason					
67. Closed Status					
68. Closed Date					
69. Closed By					
70. Closed Reason					
71. Closed Status					
72. Closed Date					
73. Closed By					
74. Closed Reason					
75. Closed Status					
76. Closed Date					
77. Closed By					
78. Closed Reason					
79. Closed Status					
80. Closed Date					
81. Closed By					
82. Closed Reason					
83. Closed Status					
84. Closed Date					
85. Closed By					
86. Closed Reason					
87. Closed Status					
88. Closed Date					
89. Closed By					
90. Closed Reason					
91. Closed Status					
92. Closed Date					
93. Closed By					
94. Closed Reason					
95. Closed Status					
96. Closed Date					
97. Closed By					
98. Closed Reason					
99. Closed Status					
100. Closed Date					

Identifiers & Preconditions

- Unique IDs support tracking and reporting
- Preconditions define required starting conditions
- Ensures tests are repeatable and traceable

Steps & Test Data

- Steps describe exact tester actions in order
- Test data provides required input values
- Each step should be clear and unambiguous

Expected Results

- Define correct system behavior precisely
- Compare actual vs. expected outcomes
- Form the basis for pass/fail decisions

Equivalence Partitioning

Groups inputs with similar behavior into partitions. Test one value per group to reduce redundant cases efficiently.

Boundary Testing

Checks values at and near the edges of valid ranges. Defects cluster at limits, making boundaries high-priority targets.

Decision Tables & State Testing

Decision tables cover combinations of rules and conditions. State testing verifies correct behavior after system transitions.

Exploratory & Risk-Based Testing

Exploratory testing combines learning with execution. Risk-based testing prioritizes scenarios most likely to cause serious failures.

TEST DESIGN TECHNIQUES

Practical Ways to Design Effective Tests

HOW DO WE CHECK THE SOFTWARE?

A step-by-step way to verify expected behavior

Simple example

TC-001 (Test case ID)

Module: Login

Test: Login with valid credentials

Steps:

1. Open login page
2. Enter valid username
3. Enter valid password
4. Click Login

Expected Result:

User reaches the dashboard.

Actual Result:

User is able to login successfully and is redirected to dashboard

Bottom takeaway

A good test case is clear, repeatable, and easy for another tester to follow.

Test ID

Unique name/number for the test.

Module

Which module is being tested(i.e. login, forget password)

Test Scenario

What needs to be tested

Preconditions

What must be ready before testing

Test Steps

What actions should the tester perform?

Test Data

What inputs or values are needed?

Expected Result

What should happen?

Actual Result

What actually happened?

Status

Pass or Fail.

Evidence

Screenshot of the findings

Scope & Objectives

- Define which cart features are included
- State what testing must prove
- Exclude out-of-scope items clearly

Test Types & Approach

- Select functional and regression tests
- Include boundary and negative cases
- Choose manual or exploratory methods

People, Tools & Environments

- Assign testers and reviewers
- Specify tools and test environments
- Confirm access and data availability

Risks & Presentation

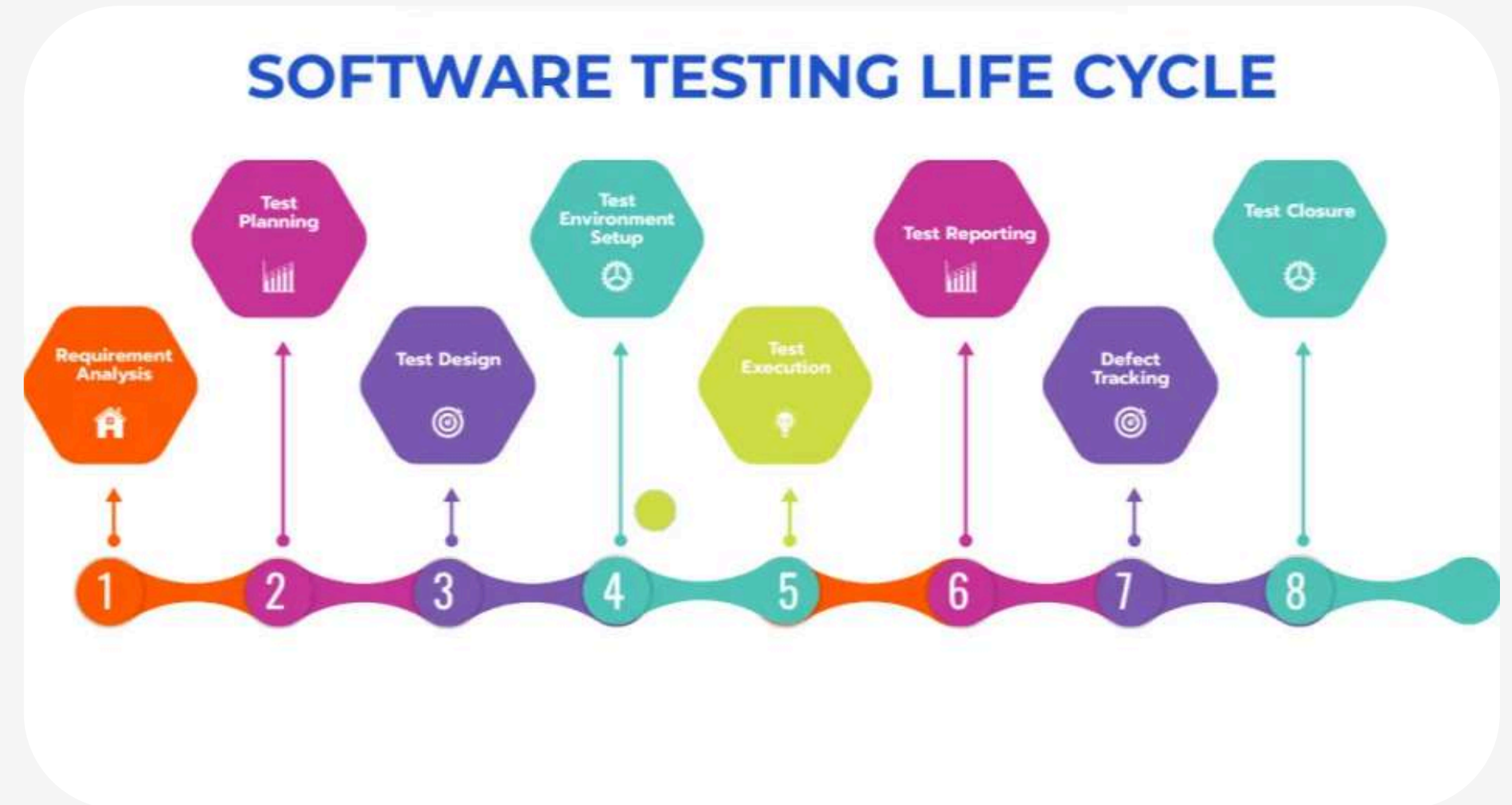
- Identify two major testing risks
- Define mitigation actions for each
- Present your plan to the group

PRACTICE TEST PLANNING

Plan & Present a Shopping Cart Test Strategy

QA LIFECYCLE

How QA Activities Progress from Requirements to Closure



Requirements & Planning

- Requirements review prevents early misunderstandings.
- Planning defines quality goals and resources.
- Design converts requirements into test coverage.

Execution & Defects

- Execution compares actual and expected behavior.
- Defect management tracks problems through resolution.
- Clear defect reports accelerate investigation.

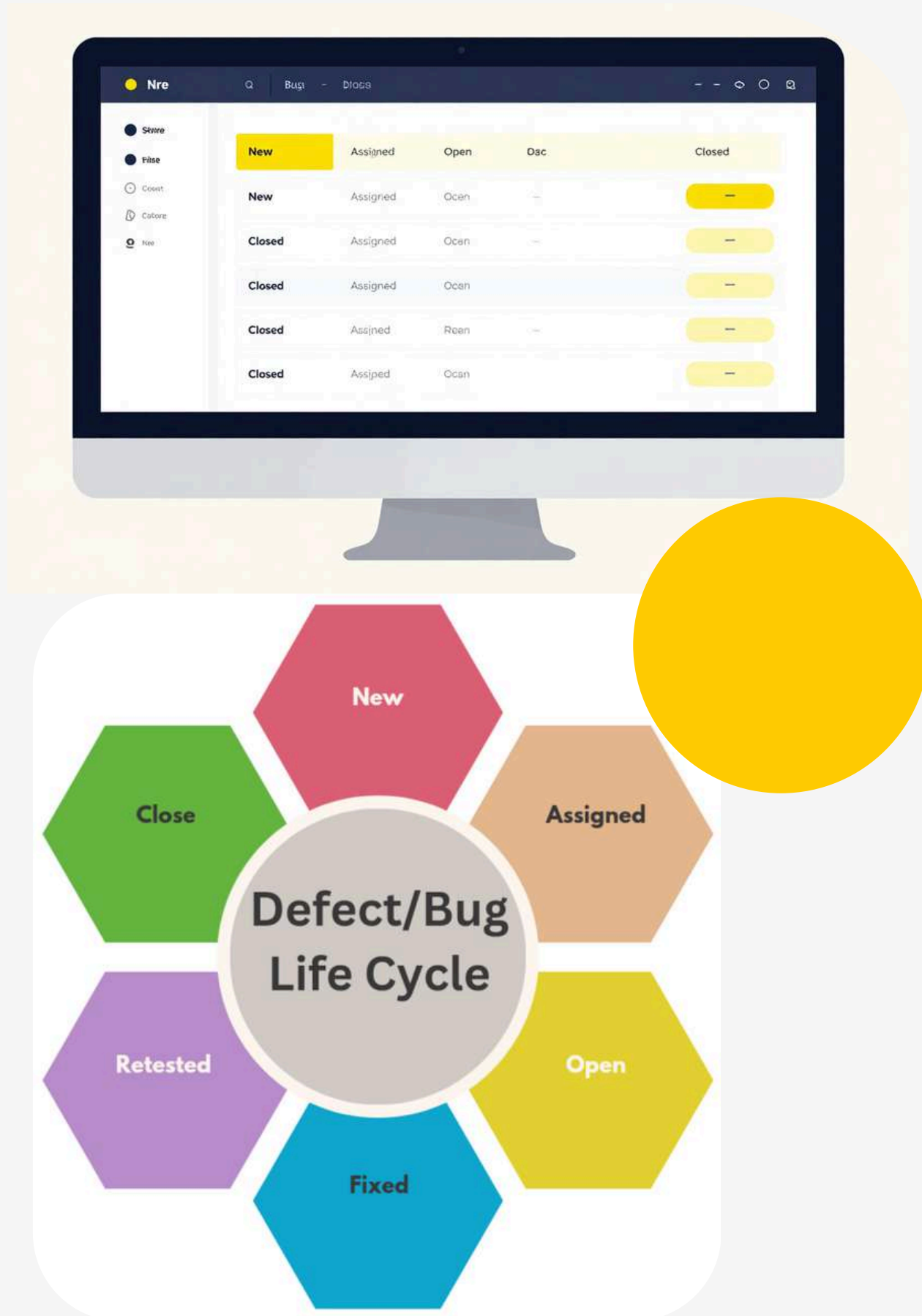
Closure & Lessons

- Closure summarizes quality, risks, and lessons.
- Regression testing checks unchanged features again.
- Each phase feeds into continuous improvement.

DEFECT LIFECYCLE

Tracking Defects Through Resolution

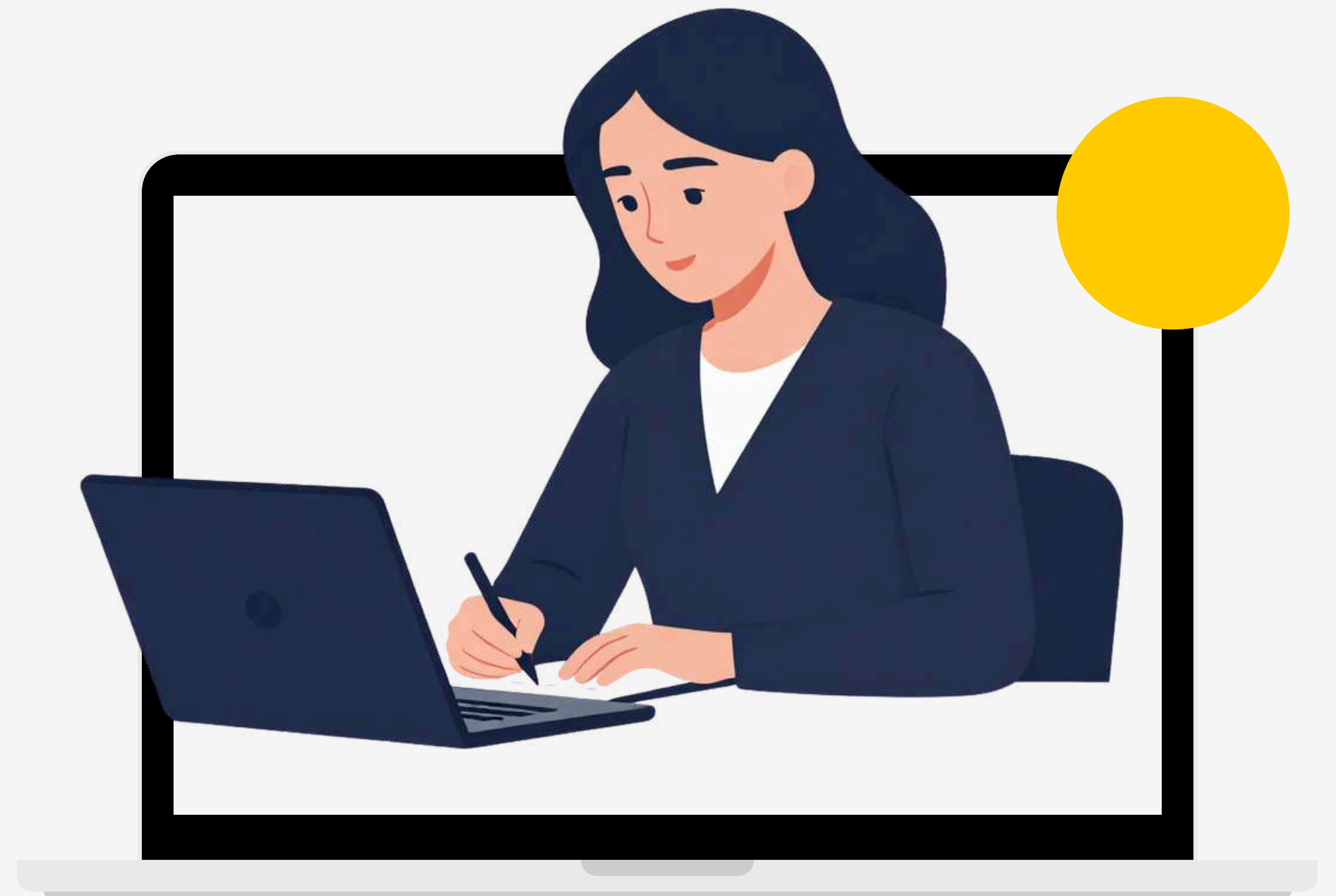
New defects await review.
Assigned defects receive responsible owners.
Open defects are investigated and fixed.
Retesting verifies the delivered correction.
Reopened defects still fail verification.
Closed defects passed verification successfully.



MANUAL TESTING

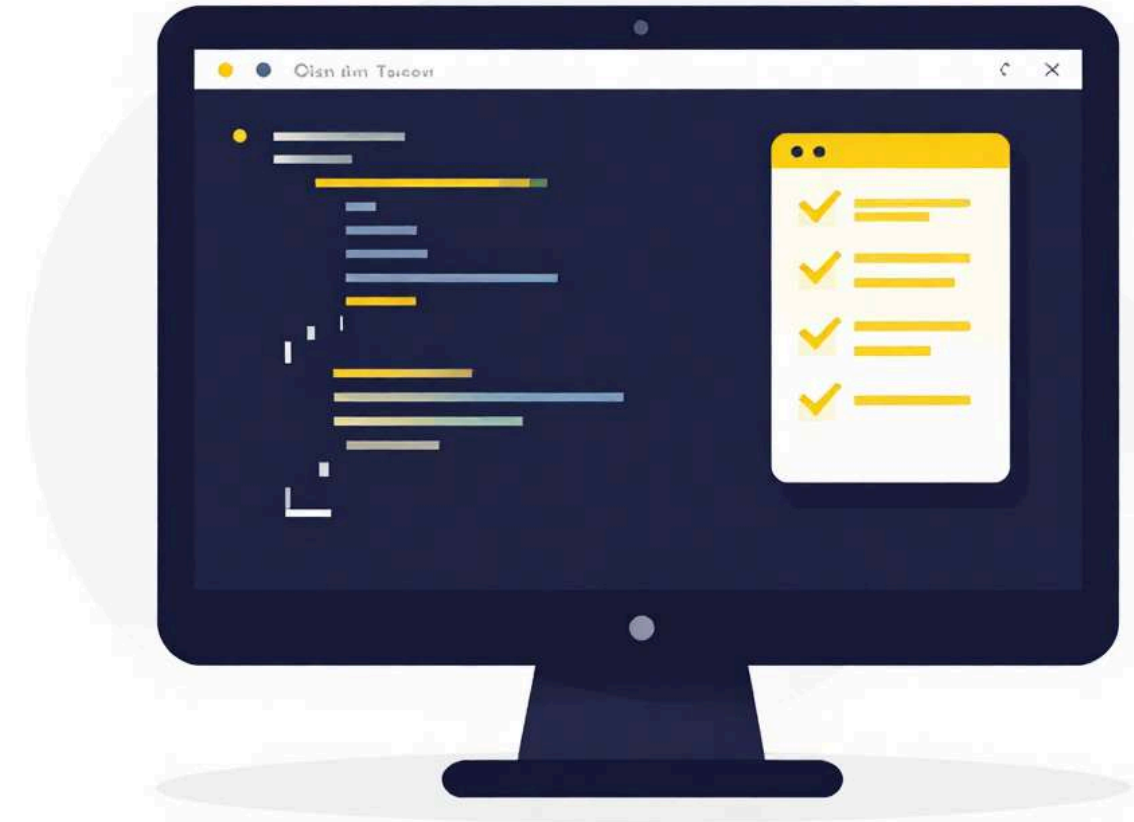
Human-Centered Verification Practices

- Manual testing uses human observation and judgment.
- Exploratory sessions investigate unfamiliar product behavior.
- Checklists support consistent repeated verification.
- Test evidence records actions and results.
- Clear defect reports accelerate investigation.
- Regression testing checks unchanged features again.



AUTOMATION BASICS

Where Automation Adds Value in QA Testing



Automate stable, repetitive, high-value scenarios first. Scripts perform actions and verify outcomes using assertions that compare actual vs. expected results.

Test data should remain controlled and reusable. Keep inputs consistent so scripts produce reliable, repeatable results across every run.

Maintenance keeps automation reliable after changes. Update scripts when features evolve to prevent false failures and ensure continued coverage.

Choose & Order

- Select one stable login scenario
- List every action in execution order
- Convert each action into pseudocode

Add Assertions

- Assert page title after login
- Assert welcome message is visible
- Assert error shown for wrong password

Run with Test Data

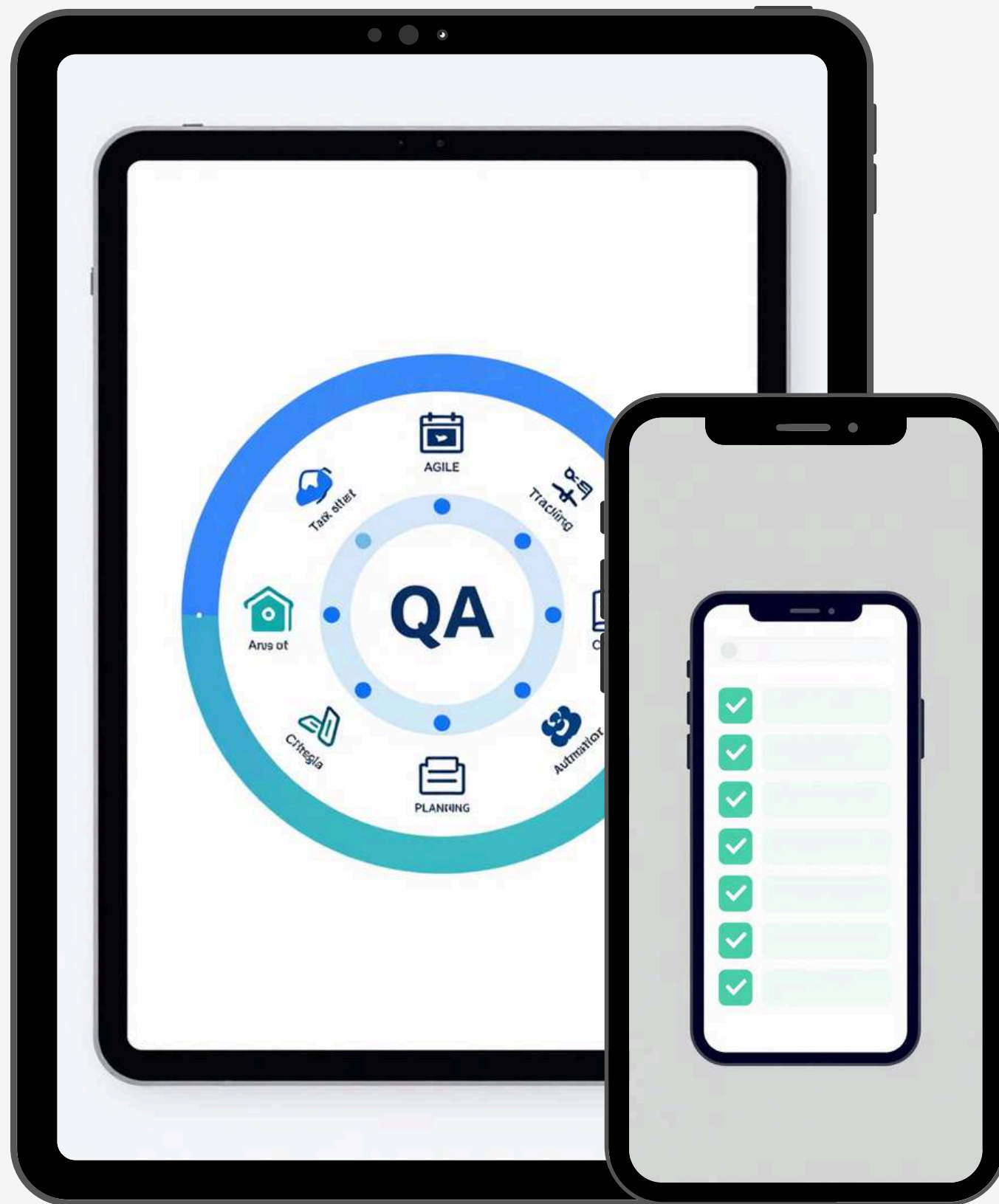
- Use valid credentials first
- Test invalid and empty inputs
- Keep test data controlled and reusable

Record & Improve

- Log each failure with detail
- Identify the root cause
- Update the script and re-run

SCRIPTING PRACTICE

Building Automated Login Checks Step by Step



KEY TAKEAWAYS

Your QA & Testing Foundation Starts Here

- Agile delivers value through short feedback cycles.
- JIRA makes work and progress visible.
- User stories connect needs with value.
- Acceptance criteria define testable success.
- Planned testing improves product confidence.
- Automation strengthens repeatable quality checks.